Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Inginerie Software și Automatică

Lucrare de laborator nr. 3

Profesor: Torică Eugeniu
Student: Zlatovcen Bogdan, grupa TI-251M

Chișinău, 2026

1 Informații generale despre Prometheus și Grafana

Laboratorul are ca scop monitorizarea unui sistem distribuit cu ajutorul Grafana și Prometheus. Sistemul monitorizat este format dintr-o aplicație Python de tip scraper, o bază de date SQLite, Prometheus, Grafana și node-exporter, toate rulate prin Docker Compose.

Prometheus este un sistem de monitorizare orientat pe metrici de tip time-series. El colectează periodic date de la endpoint-uri HTTP de tip `/metrics`, folosind un model de tip pull. Datele colectate pot fi interogate prin PromQL, ceea ce permite construirea unor expresii pentru CPU, memorie, disk, request-uri și metrici custom ale aplicației.

Grafana este un instrument de vizualizare și analiză. În laborator, Grafana folosește Prometheus ca sursă de date și afișează dashboard-ul `ADA Lab3 - 999.md Scraper`. Prin dashboard se pot urmări atât resursele nodului pe care rulează clusterul virtual, cât și indicatorii aplicației dezvoltate.

Tabela 1: Rolul serviciilor din sistemul monitorizat

Serviciu	**Rol**
`scraper`	aplicație Python care colectează anunțuri de pe 999.md și expune metrici pe portul 8000
`prometheus`	colectează metrici de la `scraper` și `node-exporter`
`grafana`	afișează dashboard-uri pe baza interogărilor PromQL
`node-exporter`	expune metrici despre nod: CPU, memorie și sistem de fișiere
`SQLite`	păstrează anunțurile deja colectate, pentru a salva doar anunțurile noi

2 Arhitectura aplicației monitorizate

Aplicația monitorizată este un scraper Python pentru categoria de mașini de pe 999.md. Deoarece pagina încarcă anunțurile prin JavaScript, scraperul folosește Playwright cu Chromium headless. La fiecare ciclu, aplicația deschide pagina, extrage linkuri către anunțuri de forma `/ro/<id>` și salvează în SQLite doar ID-urile care nu există deja.

Baza de date este un fișier SQLite aflat într-un volum Docker. Tabelul `adverts` conține câmpurile `id`, `url`, `title`, `price` și `created_at`. Periodicitatea colectării este controlată prin variabila `SCRAPE_INTERVAL_SECONDS`, cu valoarea implicită de 120 secunde.

Endpoint-ul `/metrics` al scraperului confirmă expunerea metricilor custom și a metricilor de proces. În rularea curentă, valoarea `scraper_adverts_in_db` este 2443 în screenshot-ul endpoint-ului și 2737 în raportul de rezultate după rulări suplimentare, valoare confirmată și în SQLite.

Tabela 2: Metrici expuse de aplicația scraper

Metrică	**Semnificație**
`scraper_runs_total`	numărul total de cicluri de rulare ale scraperului
`scraper_errors_total`	numărul total de erori apărute în timpul colectării
`scraper_http_requests_total`	numărul request-urilor efectuate în timpul scrape-ului
`scraper_new_adverts_total`	numărul anunțurilor noi salvate în baza de date
`scraper_adverts_in_db`	numărul total de anunțuri existente în SQLite
`scraper_scrape_duration_seconds`	durata unui ciclu de scrape, expusă ca histogramă
`process_resident_memory_bytes`	memoria RSS a procesului Python, expusă de clientul Prometheus

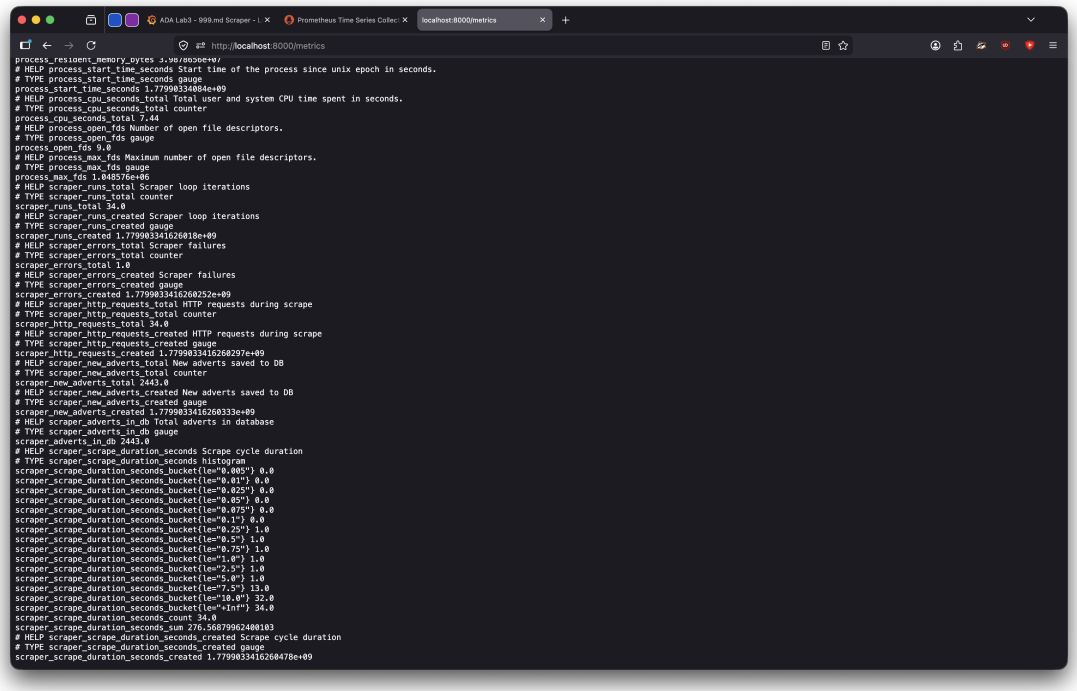


Figura 1: Endpoint-ul `/metrics` al aplicației scraper

3 Configurarea Prometheus și verificarea țintelor

Prometheus colectează metrici din două surse: `node-exporter:9100/metrics` pentru resursele nodului și `scraper:8000/metrics` pentru aplicația Python. Pagina `Status / Target health` confirmă faptul că ambele job-uri sunt active și au starea `UP`.

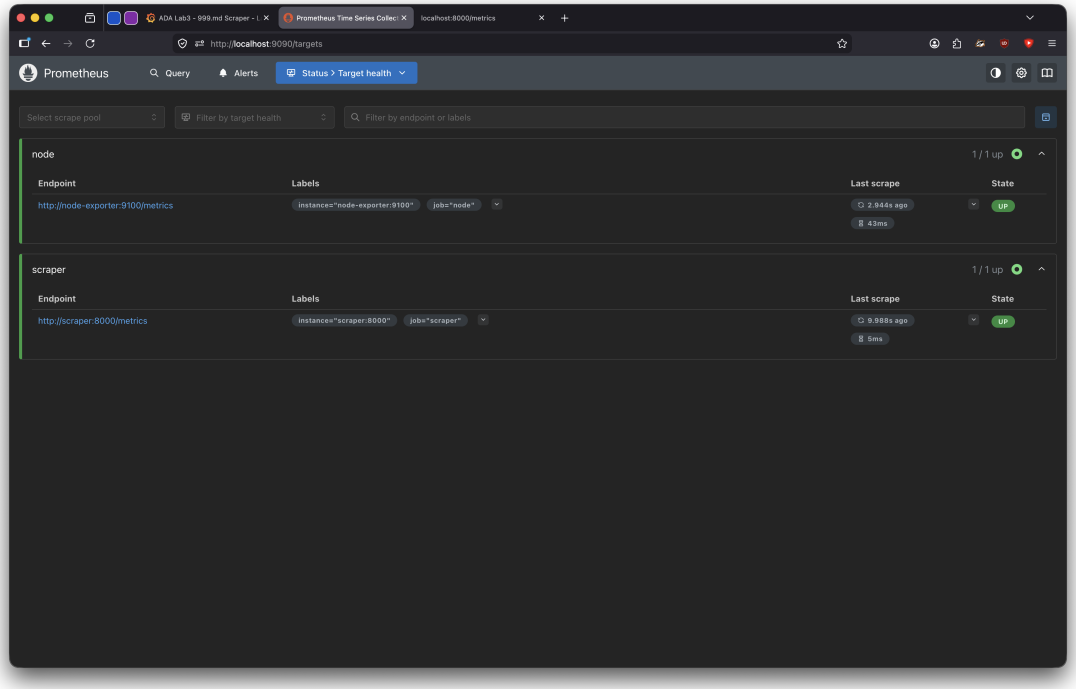


Figura 2: Prometheus Target Health: job-urile `node` și `scraper` sunt `UP`

Această verificare este importantă deoarece Grafana afișează date doar dacă Prometheus poate colecta metricile. Dacă una dintre ținte este `DOWN`, dashboard-ul poate afișa panouri goale sau valori vechi.

4 Monitorizarea CPU și a memoriei nodului

Pentru monitorizarea CPU a fost folosită metrica `node_load1`. Aceasta indică media încărcării sistemului pe ultimul minut. Graficul arată variații în timp, ceea ce confirmă că node-exporter transmite date actualizate către Prometheus.

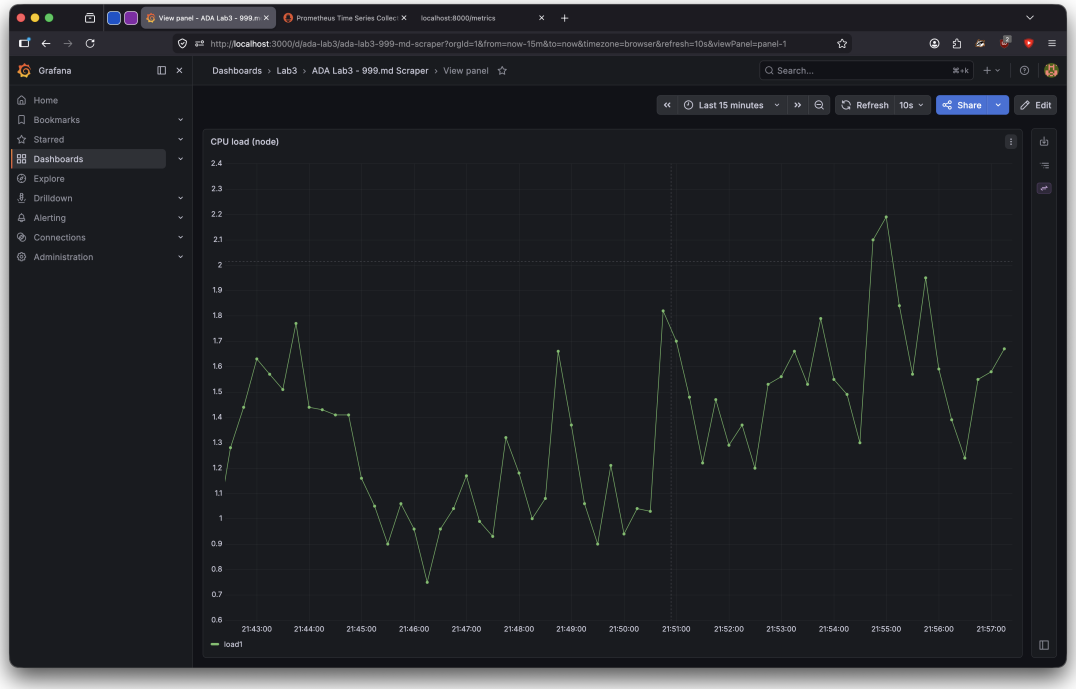


Figura 3: Dashboard Grafana: CPU load al nodului

Pentru monitorizarea memoriei ocupate a nodului s-a folosit diferența dintre memoria totală și memoria disponibilă raportate de node-exporter. Această expresie calculează memoria folosită efectiv de sistem. Graficul arată că memoria variază între aproximativ 1 GiB și 1.8 GiB în intervalul observat.

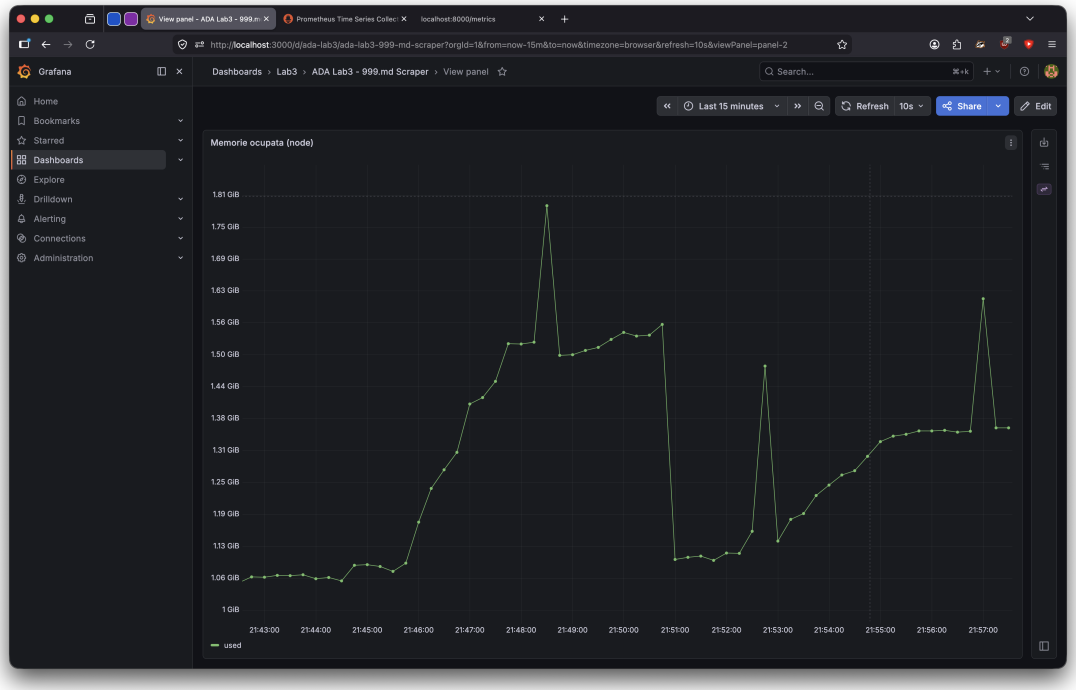


Figura 4: Dashboard Grafana: memoria ocupată pe nod

5 Monitorizarea spațiului de stocare

Pentru HDD/SSD s-a folosit metrica de filesystem din node-exporter. Expresia PromQL calculează diferența dintre dimensiunea totală și spațiul disponibil, filtrând filesystem-urile temporare sau overlay-urile Docker care ar face dashboard-ul mai greu de interpretat.

Pe macOS cu Docker Desktop, mountpoint-urile pot apărea diferit față de o instalare Linux nativă, de exemplu `/var/lib`, `/host_mnt/...` sau alte căi interne Docker. Important este că panoul primește date și prezintă evoluția spațiului folosit.

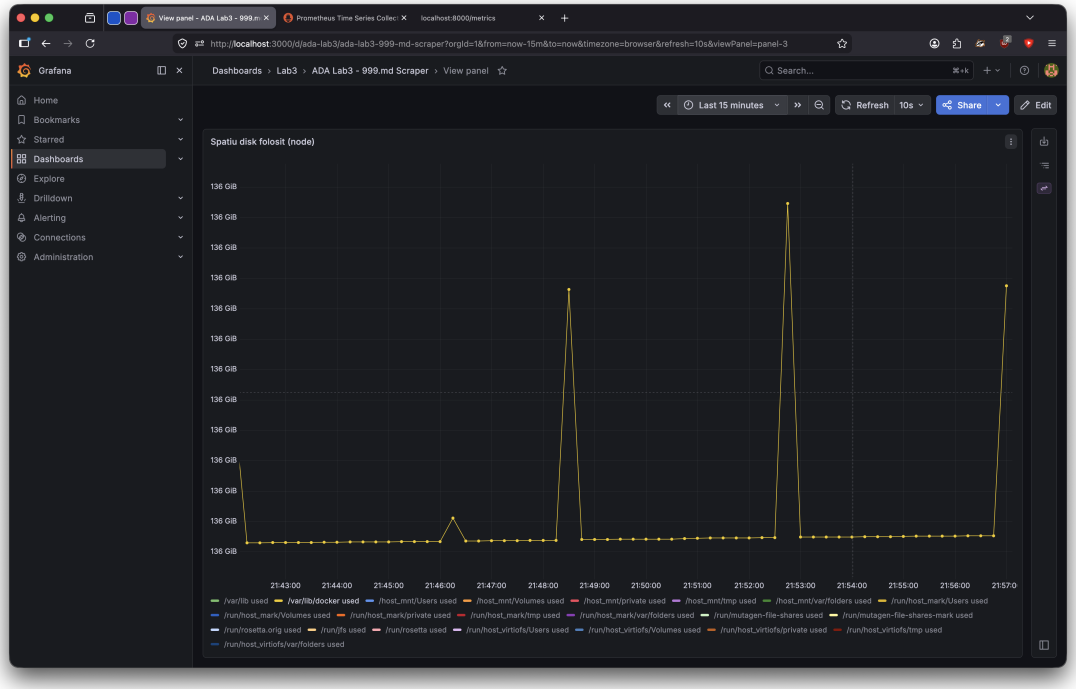


Figura 5: Dashboard Grafana: spațiu disk folosit pe nod

6 Dashboard-uri pentru aplicația web

Pe lângă metricile nodului, au fost adăugate dashboard-uri pentru aplicația scraper. Pentru request-uri pe minut a fost folosită expresia `rate(scraper_http_requests_total[1m])*60`. Graficul are valori 0 între cicluri și crește în momentele în care scraperul rulează. Acest comportament este normal, deoarece colectarea se execută periodic, la intervale de aproximativ 120 de secunde.

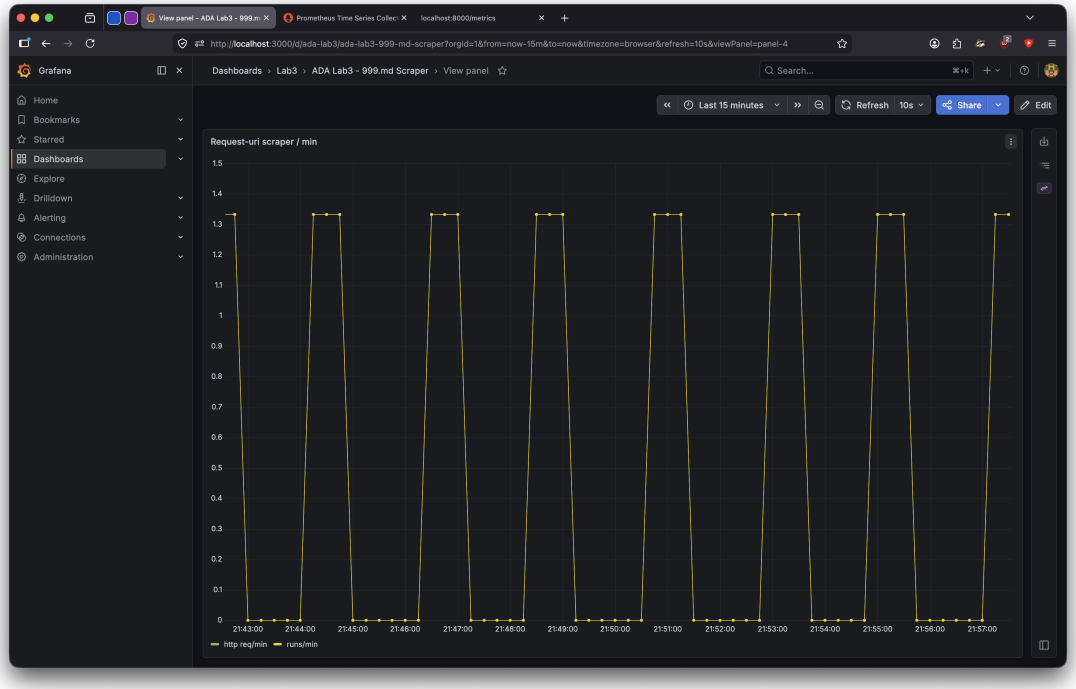


Figura 6: Dashboard Grafana: request-uri ale scraperului pe minut

Pentru memoria consumată de aplicație a fost folosită metrica `process_resident_memory_bytes`, filtrată pentru job-ul `scraper`. În screenshot, procesul scraper consumă aproximativ 38 MiB memorie RSS, ceea ce arată că Prometheus colectează și metrici de proces, nu doar metrici custom.

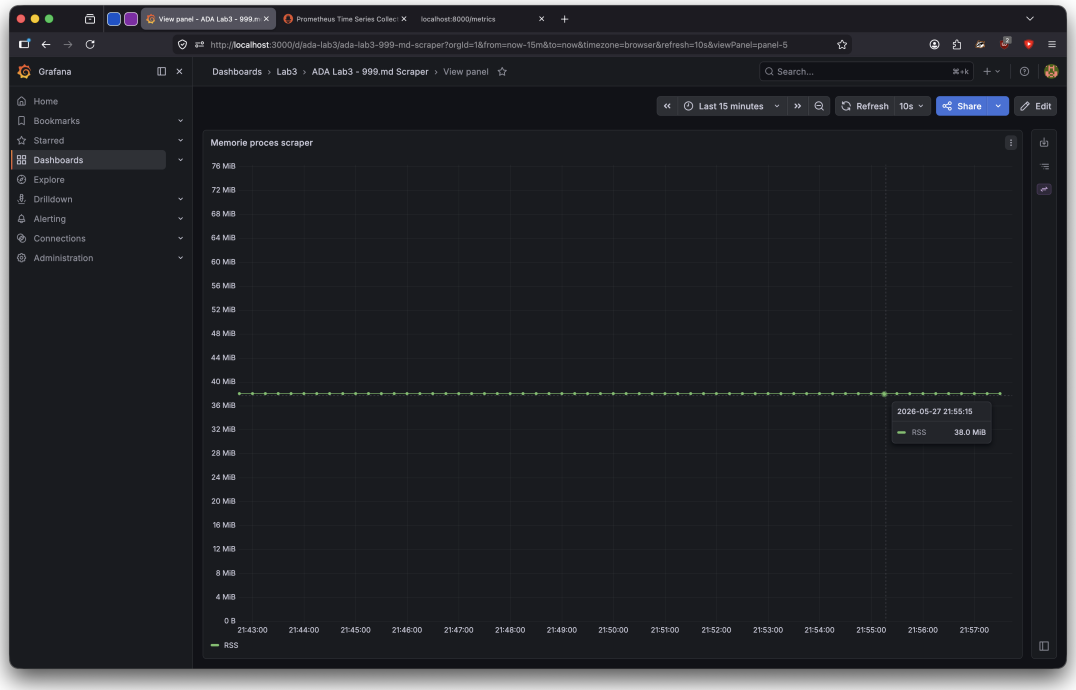


Figura 7: Dashboard Grafana: memoria procesului scraper

7 Rezultate obținute

În urma rulării sistemului, Prometheus a colectat metrici de la ambele ținte, iar Grafana a afișat panouri pentru CPU, memorie, disk, request-uri pe minut și memoria aplicației. Aplicația scraper a salvat anunțurile în SQLite și a expus metrici custom pentru numărul de cicluri, erori, request-uri, anunțuri noi și totalul anunțurilor din baza de date.

Tabela 3: Rezumatul rezultatelor monitorizate

Element verificat	**Rezultat**
Prometheus targets	job-urile `node` și `scraper` sunt `UP`
CPU nod	panoul Grafana afișează variația `node_load1`
Memorie nod	memoria ocupată este calculată din `MemTotal - MemAvailable`
Spațiu disk	panoul afișează spațiul folosit pentru filesystem-urile relevante
Request-uri aplicație	valorile cresc în timpul ciclurilor de scrape și sunt 0 între rulări
Memorie proces scraper	aproximativ 38 MiB RSS în captura analizată
Anunțuri în DB	`scraper_adverts_in_db = 2737` după rularea curentă

8 Concluzii

Laboratorul a demonstrat rolul Prometheus și Grafana în observabilitatea unui sistem distribuit. Prometheus colectează metricile prin model pull, iar Grafana le transformă în dashboard-uri ușor de analizat.

Prin node-exporter au fost monitorizate resursele nodului: CPU, memorie și spațiu de stocare. Prin endpoint-ul `/metrics` al scraperului au fost monitorizate metrici specifice aplicației: request-uri, erori, rulări, durată de scrape, numărul de anunțuri noi și totalul anunțurilor salvate în SQLite.

Soluția implementată acoperă cerințele laboratorului: informație generală despre Prometheus și Grafana, monitorizare CPU și memorie, monitorizare HDD/SSD și dashboard-uri pentru request-uri pe minut și memoria aplicației. În plus, integrarea scraperului cu SQLite arată cum metricile pot fi conectate la comportamentul real al aplicației.